

MVA Track — how to actually use this

A two-year plan from freshman engineering to M2 MVA readiness. 105 weeks, 4 tracks, 860 checkable items. This file explains how to get value out of it, and what to do about hosting.

1. The mental model

The site is **not** a to-do list you clear. It is a dependency graph with a calendar stapled to it.

Two rules explain almost every design decision:

1. **Order matters more than volume.** Nothing appears before the thing that makes it comprehensible. If you jump ahead, the later material becomes memorisation instead of understanding — which is exactly the failure mode that makes MVA courses feel impossible.
2. **Amber means load-bearing.** Items with an amber left border and a `gate` tag are prerequisites. Everything downstream silently assumes them. Skipping a normal item costs you that item. Skipping a gate costs you the next six months.

860 items over 105 weeks is roughly 8 per week. If that feels light, remember the robotics track assumes you're already spending 12+ h/week in the club — the plan schedules that time, it doesn't add to it.

2. The seven pages, and what each is for

Dashboard (/)

Your weekly home. Open this, not the roadmap.

- **Overall ring** — completion across every finishable item.
- **Per-track bars** — completion, planned h/week, and percentage per track. Watch the *gaps between* the bars, not the absolute numbers. If Math is at 40% and CS is at 12%, you are doing the comfortable track and avoiding the important one. That's the single most likely way this plan fails.
- **Current week card** — the week's tasks grouped by track, tickable inline.
- **Hours panel** — type what you actually worked, per track. Commits when you click away or press Enter. Rounds to the nearest half hour.

Before 14 Sep 2026 it shows "Week 1 begins in N days" instead of faking a current week.

Roadmap (/roadmap)

The full 105 weeks in 8 phases. Tap a phase to expand its weeks. Use this for **orientation and recovery**, not daily work — when you've lost the thread, come here to see where you are.

- **Break weeks** (French Zone C holidays) are deliberately lighter, project-focused weeks, not empty ones.
- **Summer weeks** (Aug 2027, Jul–Sep 2028) are marked optional. If you do nothing in August the plan still works; it just gets tighter afterwards.
- **TICK PHASE** / **CLEAR** bulk-toggle a whole phase. Useful when starting mid-plan or writing off a bad month honestly.

Track pages (/track/math , /cs , /ml , /rob)

The concept checklists — what each track requires you to *know*, grouped by module and ordered by dependency.

Use the strictest possible tick standard: tick a concept only when you could explain it to someone else, without notes, on a whiteboard. That is the standard MVA exams and technical interviews both apply. Ticking on "I read it and it made sense" makes the whole progress number worthless, and you'll only find out in an exam.

Each track page also states where you start and where you finish, which is worth re-reading when you feel like you're going nowhere.

Resources (/resources)

80 resources, rated 1–5 **for you specifically** — prépa maths, weak code, zero ML, aiming at MVA in Sep 2028. Not rated for fame. This is why some famous things score 3 and some obscure things score 5.

Filter by track, type, or rating. Search titles, authors, and verdicts. **MARK STARTED** tracks what you've picked up — deliberately excluded from your completion percentage, because starting a book is not progress.

Read the verdicts. Where two resources overlap, the verdict says which to actually use and why the other is still listed. Example: CS231n is rated 4 because its videos are patchy — use EECS 498 for lectures and CS231n for the assignments. That kind of thing saves you weeks.

Robotics (/robotics)

Seven builds, sequenced so each lands right after the maths that makes it non-trivial, plus a €10,000 budget tracked to the line (currently €9,757 planned, €243 slack).

Expand a project for its milestones and full BOM. The slack is intentional: something will cost more than listed, and a budget with no slack forces you to cancel a project rather than absorb a surprise.

Every build is also a portfolio artefact **and** a data source for the ML track — the same robots produce the datasets you train on. That coupling is the point.

Interviews ([/interview](#))

One milestone per phase, running continuously from Phase 2 to the end.

You asked for fluency, not competence — different skills, different training. Competence is solving the problem; fluency is solving it out loud, under time, while someone watches. Only repetition spread over years produces the second one. Two sessions a week for 100 weeks beats any four-week grind before applications.

MVA ([/mva](#))

20 courses from the real 2026 catalogue, mapped backwards: for each course, which blocks of this plan make it sittable, and the week after which you meet its prerequisites. Eight are starred — the shortlist matching a vision-plus-robot-learning profile. MVA requires 8 validated modules.

Course names and teachers drift year to year. The value here is the prerequisite mapping, not the catalogue.

3. A working rhythm that fits the plan

Weekly (~20 min, Sunday): open the Dashboard. Log the week's hours per track. Tick what you finished. Read next week's card and note which items are gates. If a gate is coming, protect time for it.

Monthly: open [/roadmap](#) and compare your per-track percentages against each other. Fix the laggard before adding anything new.

Quarterly (phase boundary): each phase ends with something showable — a repo, a benchmark, a robot doing a thing. Ship it before moving on. Check the interview milestone for that phase. Then open [/mva](#) and confirm the courses you want are still on track.

When you fall behind: you will. Don't clear the backlog item by item. Open the phase, decide what genuinely no longer matters, [CLEAR](#) it, and restart from the current week. Preserve gates ruthlessly; drop non-gates without guilt. A plan you've silently abandoned is worse than one you've consciously trimmed.

4. Progress, and where it lives

Progress is stored **server-side in a database**, not in your browser. No login — the plan is keyed to the deployment, so it follows you across laptop and phone as long as you're hitting the same instance.

Two consequences worth knowing:

- Anyone with the URL can see and modify your progress. This is fine for a personal roadmap; don't treat it as private.
- If the deployment goes away, so does the tick history, unless the database goes with it.

Four ID schemes, all stable, in case you ever export:

`w{week}-{track}-{n}` for week tasks · `c-{track}-{group}-{n}` for concepts · `car-m1` -style for robotics milestones · `iv-p1 ... iv-p8` for interview milestones.

5. Hosting — the honest version

What's running now

A dev server inside a Runnable sandbox on port 4200. That's a **preview**, not hosting: it lives as long as the sandbox does and it isn't a durable public URL. I don't control the platform's sandbox retention policy, so I can't give you a number of days — treat the preview as temporary and check the publish options in the Runnable UI for anything permanent. Publishing and custom domains are handled there, not by me.

Can this go on GitHub Pages?

Not as it stands, and the reason is structural. This is a full-stack app:

```
React + Vite frontend → Hono API (/api/rpc) → Drizzle ORM → Turso (libSQL) database
```

GitHub Pages serves **static files only**. It cannot run the Hono API or reach a database. Deploy this to Pages unchanged and you get the site rendering correctly with every tick silently failing.

You have three real options:

Option A — static build, progress in `localStorage`. I swap the database layer for browser storage. Then GitHub Pages works perfectly, hosting is free and permanent, and it's entirely outside Runnable. Cost: progress stops syncing across devices — your phone and laptop keep separate

tick histories. For a personal roadmap this is usually the right trade, and it's a change I can make in one pass.

Option B — static frontend on Pages, database elsewhere. Keep cross-device sync by pointing the frontend at a hosted Turso database directly, or at the API deployed on a free tier (Fly.io, Railway, Deno Deploy). More moving parts, a little config, but you keep every feature.

Option C — deploy the whole thing as-is to any host that runs a Bun/Node server — Fly.io, Railway, Render. Zero code changes, works exactly as it does now. Free tiers are adequate for one user.

My recommendation: Option A. This is a two-year plan and it should outlive any platform, including this one. Cross-device sync sounds valuable but in practice you'll tick from one machine on a Sunday evening. Say the word and I'll do the conversion.

Getting the code out

The project isn't a git repo yet. Whichever option you pick, step one is the same:

```
cd mva-roadmap
git init && git add -A && git commit -m "MVA roadmap"
git remote add origin git@github.com:<you>/mva-roadmap.git
git push -u origin main
```

Ask me and I'll initialise it, add a sensible `.gitignore`, and wire up the GitHub Actions workflow that builds and publishes to Pages on every push.

6. Running it locally

```
bun install
bun run db:push      # create the progress + hours tables
bun run dev          # http://localhost:4200
```

Other commands: `bun run lint`, `bun run typecheck`, `bun run build`, `bun run kill:port`.

Editing the plan

The curriculum is static TypeScript in `packages/web/src/web/content/` — no CMS, no database content. Edit these and the site follows:

File	What's in it
<code>weeks-p1/p2/p3.ts</code>	the 105 weeks. <code>!</code> prefix = gate, <code>@id</code> suffix = resource refs
<code>concepts.ts</code>	248 concepts per track
<code>resources.ts</code>	80 resources with ratings and verdicts
<code>robotics.ts</code>	7 projects, milestones, BOM lines
<code>mva.ts</code>	course map
<code>interview.ts</code>	phase milestones
<code>tracks.ts</code>	track blurbs and weekly hour budgets

One hard rule: task IDs are generated from position (`w{n}-{track}-{i}`) and are used as database keys. Adding weeks or appending tasks is safe. **Reordering or deleting existing tasks silently reassigns other items' progress.** If you must reorder, clear that week's progress first.